

CLIP: CHANGING LETTERS IN MATH PROBLEMS

Anonymous authors

Paper under double-blind review

ABSTRACT

Large language models (LLMs) have demonstrated strong capabilities in solving mathematical problems. However, it remains unclear to what extent their reasoning is based on abstract logical structures versus reliance on surface-level patterns from their training data. To investigate this, we propose a simple yet effective adversarial evaluation method: randomly substituting variable names in math problems while preserving their mathematical equivalence.

We apply this method to multiple datasets. Our experiments on a spectrum of models—from small specialized models (e.g., Qwen2.5-Math-1.5B) to state-of-the-art reasoning models (e.g., DeepSeek-V3.2-Thinking)—reveal a clear divergence. Our findings suggest that robustness against purely syntactic perturbations is not a given but an emergent property scaling with model capability, providing a new perspective on evaluating the true reasoning depth of LLMs.

1 PROJECT INFORMATION

1.1 CONTRIBUTIONS

Idea: Bangjie Xu and Haoren Luo

Dataset Construction: Zibo Diao designed the prompt strategy and generated and cleaned the adversarial datasets for MATH500 and AIME benchmarks. Bangjie Xu manually collected and made up 49 math problems.

Building The Evaluation Pipeline: Junhao Huang.

Deploying and Evaluating Models: Bangjie Xu and Haoren Luo.

Report Writing: All four members.

(Project Leader: Bangjie Xu)

1.2 AI USING

During coding, we made use of Copilot. We also used some LLM to polish the English in our reports.

1.3 DURING THE RESEARCH PROCESS, WE DID NOT RECEIVE SUPPORT FROM THE PI.

1.4 AUTODL SERVICE USING

We mainly used AutoDL to deploy and evaluate five small models. The usage is around 400 RMB. We mainly used RTX 4090.

We also used the api of DeepSeek on <https://ai.paratera.com/#/cloud/compute>, which is also supported by Tsinghua. (Every student of Tsinghua University is entitled to a certain amount of free quota.) The usage is around 100 RMB.

2 INTRODUCTION

We aim to address the following question: How close are current large language models (LLMs) to formal verifiers in the field of mathematical problem-solving? We propose a method for generating adversarial samples: replacing letters in test questions while preserving the original meaning of the problems. We observe that such replacement leads to a decrease in the accuracy rate of LLMs, however for real formal verifiers, there should be no affect.

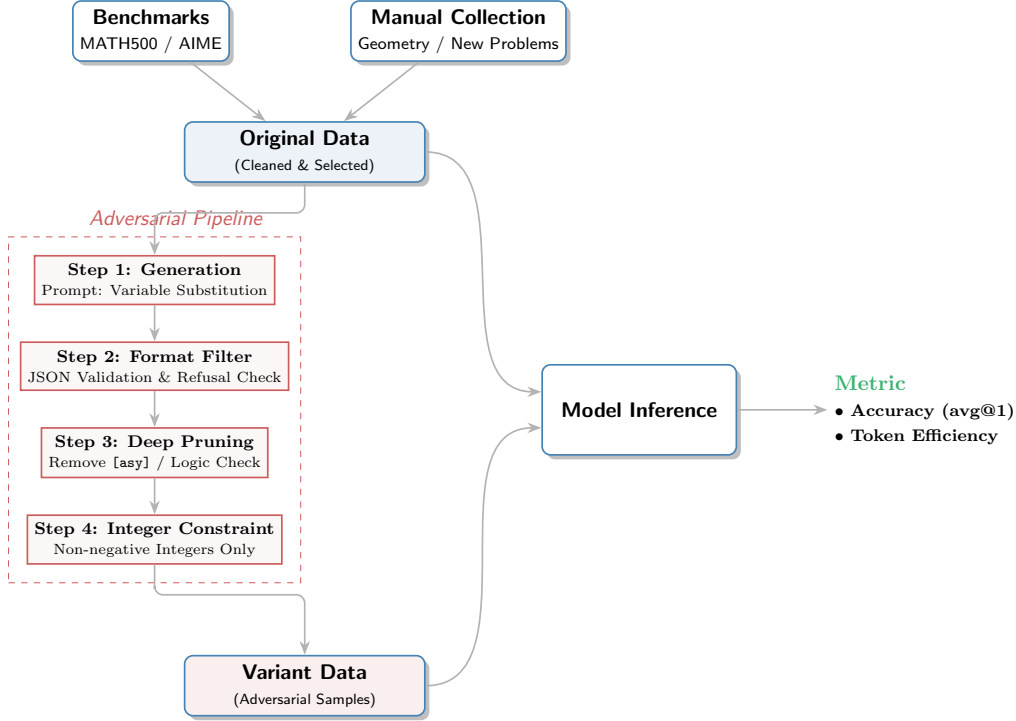


Figure 1: The overview of our adversarial evaluation pipeline. We generate variant data via strict variable substitution, filter them through a multi-stage process, and compare model performance against the original benchmarks.

3 RELATED WORK

3.1 LLM REASONING AND ROBUSTNESS

Recent advancements in Large Language Models (LLMs) have significantly improved mathematical reasoning, evolving from Chain-of-Thought (CoT) prompting (Wei et al., 2022) to specialized models using synthetic fine-tuning and reinforcement learning (Yang et al., 2024; DeepSeek-AI et al., 2025a). Despite these gains, robustness remains a challenge. General adversarial attacks in NLP typically employ character-level noise or semantic paraphrasing (Jia & Liang, 2017), but these are often ill-suited for mathematical tasks where precision is paramount.

3.2 ADVERSARIAL ATTACKS ON MATH PROBLEMS

For math problems, recent work aims at generating similar or equivalent problems on which the models have lower accuracy than the original problems. They focus on the following approaches: (1) Modifying numbers;(Xie et al., 2024) (2) Adding distracting conditions;(Yang et al., 2025) (3) Modifying words;(Zhou et al., 2023) (4) Rewriting equations.(Luong et al., 2025)

4 MOTIVATION: SYMBOLIC INVARIANCE

We noticed that existing methods to generate adversarial samples in math problems all have drawbacks. (1) and (3) may not apply to Olympic level problems (e.g. IMO, CMO) which may not contain numbers and "non-logical" words. (1), (2) and (4) may increase the computational complexity, which influences the difficulty of the problem itself. Also, these methods may require the attacker to have certain math problem solving ability.

To overcome these limitations, we generate samples that only modify the variable names in math problems. This ensures that the problems are mathematically equivalent. An example is shown below.

Original: The coordinates of a parallelogram are (5, 3), (6, 8), (7, 4) and (x, y) and $x > 7$. What is the value of $x + y$?

Variation: The coordinates of a parallelogram are (5, 3), (6, 8), (7, 4) and (C, f) and $C > 7$. What is the value of $C + f$?

This method overcomes previous limitations by: (1) Preserving exact logical equivalence and difficulty; (2) Being applicable to abstract (non-numerical) domains like geometry proofs; and (3) Eliminating the need for a solver to re-calculate answers, as the logic remains identical.

5 METHODOLOGY

To evaluate the robustness of LLMs against symbolic perturbations, we design a pipeline that generates adversarial samples by substituting variable names while strictly preserving mathematical equivalence.

5.1 ADVERSARIAL GENERATION PIPELINE

We utilize `gpt-oss-120b` (OpenAI, 2024) as the generator due to its superior instruction-following capabilities. The generation process follows a strict protocol:

1. **Variable Mapping:** For each problem, we generate a random permutation of characters from $[A - Z, a - z]$. The prompt instructs the model to map the i -th unique mathematical variable in the text to the i -th character in the permuted list.
2. **Constraints:** The model is explicitly forbidden from performing reasoning, paraphrasing, or modifying numerical constants and operators.

To ensure the quality and validity of the generated samples, we apply a four-stage filtration process:

1. **Refusal & Format Check:** Discarding outputs where the model refuses generation or produces invalid JSON.
2. **Visual Pruning:** Removing problems containing geometric code (e.g., Asymptote), as textual substitution creates mismatches with static diagrams.
3. **Sanity Check:** Using heuristics to detect hallucinations, including residual imaginary units (i) and empty variant IDs.
4. **Integer Constraint:** We only use problems whose answers are non-negative integers in order to avoid discrepancies in accuracy resulting from the marking process.

5.2 DATA ATTRITION ANALYSIS

The rigorous nature of our pipeline resulted in significant data attrition, prioritizing quality. The detailed funnel is shown in Table 1.

Ultimately, we obtained 135 MATH500 pairs and 491 AIME pairs of high fidelity. We have manually checked that all the 135 MATH500 pairs are mathematically equivalent and legal. After the pipeline, problem variants in AIME dataset still have mistakes, so we manually picked 51 correct pairs.

Table 1: Data Attrition Funnel: Retention rates across pipeline stages.

Pipeline Stage	MATH500	Retention	AIME	Retention
0. Initial Dataset	500	100%	933	100%
1. Augmentation Success	293	58.6%	817	87.6%
2. Visual & Number Filter	222	44.4%	776	83.2%
3. Deep Pruning (Structure)	208	41.6%	491	52.6%
4. Integer Filter (Final)	135	27.0%	491	52.6%

6 EXPERIMENTS AND RESULTS

6.1 SETUP

6.1.1 DATASETS

We evaluate models across four datasets representing different domains and difficulty levels. All samples only contain non-negative integer answers.

- **MATH500-Subset:** A filtered subset of 135 problems from the MATH500 benchmark (Lightman et al., 2024).
- **AIME-Subset:** A challenging subset of problems derived from the AIME dataset (Neubig, 2024).
- **Geometry-Custom:** 30 text-only geometry problems collected from recent Chinese High School Entrance Exam (Zhongkao) mock papers. Since some models do not have multimodal understanding capabilities, the geometry problems we selected either contain no images or do not require the use of images for their solutions.
- **Contamination-Free Set:** 19 newly constructed AIME-level problems to rule out the effects of training data contamination.

6.1.2 MODELS

We select a diverse array of models ranging from efficient specialized models to state-of-the-art reasoning engines: **Qwen2.5-0.5B-Instruct**, **Qwen2.5-Math-1.5B-Instruct** (Yang et al., 2024), **DeepSeek-R1-Distill-Qwen-1.5B** (DeepSeek-AI et al., 2025a), **Ministral-3-3B-Instruct** (Mistral-AI, 2025), **Qwen3-4B-Thinking** (Team, 2025), and **DeepSeek-V3.2-Thinking** (DeepSeek-AI et al., 2025b).

6.1.3 EVALUATION METRICS AND PROTOCOL

We use avg@1 as the metric for accuracy.

To measure token efficiency, we calculate the Average Token Count (TC) for successful attempts. Let o_i and v_i be the number of correct samples for the original and variant versions of problem i , respectively. t_{ij} is the token count of the j -th correct attempt in the i -th problem. The metric is defined as:

$$TC_{\text{original}} = \frac{\sum_{i=1}^p 1[o_i v_i > 0] \frac{\sum_{j=1}^{o_i} t_{ij}}{o_i}}{\sum_{i=1}^p 1[o_i v_i > 0]}, \quad TC_{\text{variant}} = \frac{\sum_{i=1}^p 1[o_i v_i > 0] \frac{\sum_{j=1}^{v_i} t_{ij}}{v_i}}{\sum_{i=1}^p 1[o_i v_i > 0]}$$

Inference is accelerated using **vLLM** (Kwon et al., 2023) with batch processing. Answer extraction prioritizes the final continuous digit string in the output.

6.2 RESULTS

6.2.1 SUBSET OF MATH500

For Qwen/Qwen2.5-0.5B-Instruct, each problem is tested 20 times, for Qwen/Qwen2.5-Math-1.5B-Instruct, each problem is tested 50 times, for the other three models, each problem is tested 10 times. We observed that all the five models have accuracy drop under our attacks. Ministral-3-3B has the most significant drop. For Qwen2.5-0.5B-Instruct, the drop is smallest.

Table 2: Small models’ performance degradation on MATH500

Model	Original	Variant	Difference
Qwen/Qwen2.5-0.5B-Instruct	23.78	21.93	-1.85
Qwen/Qwen2.5-Math-1.5B-Instruct	77.02	72.01	-5.01
Qwen/Qwen3-4B-Thinking-2507	89.63	86.15	-3.48
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	72.89	68.89	-4.00
mistralai/Minstral-3-3B-Instruct-2512	84.07	78.22	-5.85

6.2.2 GEOMETRY-CUSTOM DATASET

For each model, each problem is tested 50 times, so there are 1500 data points. We observed that all the five models have accuracy drop under our attacks. Qwen2.5-Math-1.5B-Instruct has the most significant drop. For Qwen2.5-0.5B-Instruct, the drop is smallest, this is because this model may be too weak to solve geometry problems in our dataset.

Table 3: Small models’ performance degradation on 30 geometry problems

Model	Original	Variant	Difference
Qwen/Qwen2.5-0.5B-Instruct	9.53	9.07	-0.46
Qwen/Qwen2.5-Math-1.5B-Instruct	32.67	25.20	-7.47
Qwen/Qwen3-4B-Thinking-2507	76.80	75.67	-1.13
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	44.80	38.13	-6.67
mistralai/Minstral-3-3B-Instruct-2512	51.60	48.13	-3.47

6.2.3 SUBSET OF AIME

On a subset of 51 AIME problems, each problem was tested for 8 times. The TC (token count) metric is defined in previous paragraph. We did not observe any meaningful changes in accuracy or the number of tokens.

Table 4: DeepSeek-V3.2-Thinking’s accuracy and average token count on AIME problems

Model	Metric	Original	Variant	Difference
DeepSeek-V3.2-Thinking	Accuracy(avg@1)	84.31%	84.80%	0.49%
DeepSeek-V3.2-Thinking	Average Token Count	10603	10512	-0.99%

6.2.4 SET OF NEW PROBLEMS

On a subset of 19 brand new AIME level problems, each problem was tested for 16 times for DeepSeek-V3.2-Thinking and 40 times for Qwen3-4B-Thinking-2507. The accuracy metric is avg@1. We did not observe any meaningful changes in accuracy.

Table 5: Two models’ performance on 19 brand new AIME level problems

Model	Original	Variant	Difference
DeepSeek-V3.2-Thinking	65.46	65.79	0.33
Qwen3-4B-Thinking-2507	25.25	24.25	-1.00

7 ANALYSIS AND DISCUSSION

In this section, we analyze the performance degradation observed across different models and datasets. Our goal is to determine whether LLMs perform reasoning based on abstract logical structures—similar to formal verifiers—or if they rely on surface-level pattern matching derived from training data distributions.

7.1 SENSITIVITY TO NOTATIONAL CONVENTIONS

The core premise of our experiment is that a rigorous mathematical reasoner should be invariant to symbol permutation (e.g., solving for x is logically identical to solving for $\sigma(x)$). However, our results on MATH500 and the self-made geometry dataset show a consistent performance drop across most small-to-medium sized models when variable names are perturbed.

We hypothesize that this fragility stems from the **distributional bias** in pre-training data. Mathematical textbooks and corpora follow strict conventions (e.g., using n for integers, x for unknowns, $\triangle ABC$ for triangles). When these are replaced with unconventional symbols, the models’ “approximate retrieval” mechanisms conflict with their logical reasoning engines. This indicates that for these models, mathematical ability is partially entangled with the memorization of familiar templates.

7.2 SCALING LAWS AND EMERGENT ROBUSTNESS

While smaller and specialized models exhibited clear fragility, our evaluation of the SOTA model, **DeepSeek-V3.2-Thinking**, revealed a distinct behavioral shift. Remarkably, this model demonstrated negligible performance degradation between the original and variant datasets, maintaining near-identical accuracy regardless of symbol permutation.

This observation suggests a crucial extension to standard scaling laws (Kaplan et al., 2020): scaling parameters and inference-time compute (via “Thinking” or extended Chain-of-Thought) yields not only higher accuracy but also **emergent robustness**. We hypothesize that sufficiently large and reasoning-heavy models transition from “surface-level pattern matching” to “abstract semantic reasoning.” In this regime, the model successfully disentangles the mathematical logic from its symbolic representation, effectively treating variables as abstract placeholders. This suggests that robustness against notation attacks is a phase-transition property that emerges only after a certain threshold of reasoning capability is reached.

8 CONCLUSION

We have observed that large language models (LLMs) do possess a certain degree of formal reasoning capability — they do not merely memorize and apply mathematical formulas; instead, they actually comprehend the relationships between variables in different positions when using these formulas.

For small ($\leq 4B$) models, modifying the letters in the question led to a decrease in accuracy. This might be due to a mismatch in the training data or insufficient representational capacity.

For large language models, the modules within the model (such as the attention mechanism) have a strong ability to utilize the logical relationships between tokens at various positions (e.g., the quantitative relationship between two variables (tokens)). Furthermore, during training, these models may learn more about the logical relationships of variables in mathematical problems based on abstract meanings, rather than just the variable names.

ACKNOWLEDGMENTS

We would like to express our gratitude to Tianxing He and Jingzhao Zhang for their insightful discussions and valuable suggestions during the initial stages of this project. We also thank the support provided by the computing resources of Tsinghua University.

REFERENCES

- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- DeepSeek-AI, Wenfeng Liang, Damai Dai, Daya Guo, Dejian Yang, et al. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025b.
- Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. *arXiv preprint arXiv:1707.07328*, 2017.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Karl Cobbe, and Jeany Hilton. Let’s verify step by step. In *Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning, 2025. URL <https://arxiv.org/abs/2511.01846>.
- Mistral-AI. Ministral 3: Efficient language models for the edge (v2512). <https://mistral.ai/news/ministraux-v3/>, 2025. Released: 2025-12-02.
- Graham Neubig. AIME 1983-2024 dataset. <https://huggingface.co/datasets/gneubig/aime-1983-2024>, 2024. Accessed: 2025-01-01.
- OpenAI. gpt-oss-120b: A large-scale open-source language model. <https://github.com/openai>, 2024. Accessed: 2025-01.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 24824–24837, 2022.
- Roy Xie, Chengxuan Huang, Junlin Wang, and Bhuwan Dhingra. Adversarial math word problem generation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 5075–5093, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.292. URL <https://aclanthology.org/2024.findings-emnlp.292/>.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5-math technical report: Toward mathematical expert model via synthetic data and iterative processing. *arXiv preprint arXiv:2409.12122*, 2024.

Kaiqi Yang, Hang Li, Yucheng Chu, Zitao Liu, Mi Tian, and Hui Liu. Iterative llm-based generation and refinement of distracting conditions in math word problems, 2025. URL <https://arxiv.org/abs/2510.08615>.

Zihao Zhou, Qiufeng Wang, Mingyu Jin, Jie Yao, Jianan Ye, Wei Liu, Wei Wang, Xiaowei Huang, and Kaizhu Huang. Mathattack: Attacking large language models towards math solving ability, 2023. URL <https://arxiv.org/abs/2309.01686>.

A FAILURE ANALYSIS IN DATASET CONSTRUCTION

A.1 SMALL MODELS

- **Instruction Adherence vs. Robustness Trade-off:** With loose prompts, smaller models tended to "over-modify," frequently hallucinating by converting numerical constants (e.g., "3") into variables from the replace list. Conversely, when strict constraints were applied, these models became overly conservative, refusing to modify valid questions.
- **Format Pollution:** Reasoning-enhanced models (e.g., DeepSeek-R1) often leaked their "Chain of Thought" (internal reasoning traces) into the output, corrupting the JSON structure required for the pipeline.

A.2 FAILURE ANALYSIS OF OUR FINAL PIPELINE

- **MATH500:** The dataset experienced two major drops. First, 41.4% were rejected during generation due to complex LaTeX structures triggering model refusal. Second, the final **Integer Filter** removed an additional 73 problems (dropping from 208 to 135), as MATH500 contains many questions with fractional, irrational, or interval answers.
- **AIME:** While generation was robust, the AIME dataset suffered a massive drop in the **Deep Pruning** stage (Empty Variant Check), losing 285 IDs due to context-length truncations on long problem statements. Notably, the final **Integer Filter** caused zero loss (491 $\rightarrow$ 491), confirming the dataset's inherent property of non-negative integer answers.

B AUGMENTATION SCRIPT AND PROMPT ENGINEERING

The following script contains the rigorous system prompt used in our experiments. To preserve the exact experimental setting, we present the prompt in its original language (Chinese). The escapeinside mechanism is used to render CJK characters within the Python block.

```
1 import random
2 import json
3
4 target_model = "openai/gpt-oss-120b"
5 candidate_chars = list("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz")
6
7 def build_prompt(problem):
8     """
9     Constructs the prompt with a randomized substitution list.
10    """
11    replace_list = random.sample(candidate_chars, 20)
12
13    # Note: The text inside ... is rendered by CJKutf8 package
14    prompt_content = f"""
15 你是一个严格的数学变量替换系统。你的任务是对题目执行【字符级别的强制替换】，不允许做任何推理或改写。
16
17  【极其重要：必须遵守以下约束】
18  1. 你只能替换\数学变量"，包括：
19     - 26 English letters (a-z, A-Z)
20     - 常见希腊字母 ( $\alpha\beta\gamma\delta\dots\omega$ )
21  2. 数学变量必须严格按照它们【在题目中第一次出现的顺序】进行编号：
22     第1 个变量  $\rightarrow$  replace_list[0]
23     第2 个变量  $\rightarrow$  replace_list[1]
24     ...
25  3. 同一个原始变量出现多次时，必须替换成同一个新字符。
26  4. 不允许修改以下内容：
27     - 中文、英文单词（如Angela、height、area 等）
```



```
432 28 - 数字、单位、运算符
433 29 - 任何非变量字符
434 30 - 整句结构、换行格式
435 31 5. 不允许改写题目，不允许添加额外解释。
436 32 6. 若题目中完全不存在数学变量，则必须输出：
437 33 无法更改题目
438 34
439 35 【输出格式要求（必须严格遵守）】
440 36 你必须只输出：
441 37 1. 完整替换后的题目文本（不可省略原本换行）
442 38 或
443 39 2. \无法更改题目"
444 40
445 41 【替换列表】
446 42 replace_list = {replace_list}
447 43
448 44 【现在开始执行】
449 45 下面是题目：
450 46
451 47 {problem}
452 48 """
453 49 return prompt_content, replace_list
```

Listing 1: Core Generation Logic with Original Prompt

C MORE DATASET INFORMATION

Dataset	Language	Source	# Of Problems Tested
MATH500	English	Lightman et al. (2024)	135
AIME	English	Neubig (2024)	51
Geometry-Custom	Chinese	Collected from questions compiled by others	30
Contamination-Free Set	Chinese	Hand-made	19